

Bio-Inspired Recursive Computation 1.0

Deterministic Adaptation Models and the Boundary Between Metaphor, Simulation, and Biology

Shafaet Brady Hussain | Independent researcher, ResearchForumOnline | 18 August 2026

Abstract

I consolidate the bio-digital and DNA-themed research corpus around a reproducible software claim: biological language can inspire computational structures, but an analogy is not evidence of biological mechanism. The release audits a frozen “Genetic Adaptation Simulator” and supplies a separate bounded recursive adaptation reference. The frozen program evaluates independent points of a scalar formula; it does not implement inheritance, population selection, mutation, genotype, phenotype, fitness, DNA storage, wetware, or recursion. Five deterministic audit findings reproduce an unused y parameter, pointwise rather than recursive evaluation, collapsed invalid-log inputs, a step discontinuity at zero, and possible exponential overflow. The new recurrence passes determinism, closed-form, boundedness, monotonicity, and error-contraction checks. These are software results only. No clinical efficacy, genetic interpretation, biological experiment, consciousness, healing, or independent validation is claimed.

1. Research question and contribution

What remains defensible when a corpus uses “DNA,” “genetic,” “organic,” and “biomorphic” language to describe computation? The answer is a layered boundary. A metaphor may motivate a design; a mathematical model defines variables and transformations; a software simulation executes that model; a biological claim additionally requires correspondence with observed biological processes and suitable experimental evidence.

The contribution is therefore twofold: an exact semantic audit of the available program, and a minimal recurrence whose behavior is analytically checkable. The earlier Bio-Digital Research Boundaries draft is consolidated here rather than published as a second weak paper.

2. Frozen Genetic Adaptation Simulator

The inspected Python file defines

$$G(x) = b_2 \log(b_1 + \eta Q x) \exp(\lambda x) [1 + \alpha H(-x) + \beta H(x) + \gamma \exp(-\theta Q x^2)].$$

The parameter y appears in the function signature but is unused. Inputs are converted to a list and evaluated independently. Values with a non-positive logarithm argument are replaced by $1e-12$ rather than rejected. The named delta terms are implemented as Heaviside step indicators, not Dirac delta distributions. These details are important because the README describes systemic learning and

adaptation while the executable object is a deterministic curve generator.

The formula may still be explored as a piecewise nonlinear response surface. It is not, on present evidence, a genetic algorithm. Genetic algorithms conventionally require a population of candidate representations, variation, evaluation, and selection across generations [1]. None of those mechanisms exists in the frozen simulator.

3. Reproducible semantic audit

An independent exact mirror avoids redistributing or importing the application file. The audit reproduces five properties:

1. Changing y from -999 to 999 leaves every output unchanged.
2. Evaluating a vector equals evaluating each point separately, demonstrating no state transfer or recursion.
3. With neutral growth and decay parameters, distinct invalid-log inputs collapse to the same clamped logarithm value.
4. At zero neither step activates, so configured side weights create a discontinuity relative to an arbitrarily small positive input.
5. A sufficiently large positive λx raises an overflow exception.

These results falsify descriptions of the frozen program as recursive learning, evolution, or robust numerical simulation. They do not falsify its use as an artistic or exploratory formula when its domain and limitations are explicit.

4. Bounded recursive reference

To make “recursive adaptation” testable, the package defines the transparent recurrence

$s_{t+1} = s_t + \mu (\text{target} - s_t)$, with $0 < \mu \leq 1$.

For initial state 0, target 1, and $\mu = 0.2$, the closed form is $s_t = 1 - 0.8^t$. After 20 steps the released run reaches approximately 0.98847. The verifier confirms deterministic reruns, agreement with the closed form within $1e-12$, bounded values in $[0,1]$, monotone increase, and contraction of target error.

This recurrence has memory because the next state depends on the prior state. It is inspired by adaptation in the ordinary engineering sense of iterative adjustment. It is not a model of genes, cells, organisms, natural selection, learning in humans, or biological fitness.

5. Relationship to biological and molecular computing

Bio-inspired computation and biological computation are not synonyms. Holland’s work formalized adaptive systems and became foundational to genetic algorithms [1]. Adleman’s laboratory demonstration used DNA molecules to solve a small combinatorial instance, establishing a genuine molecular-computing experiment [2]. Those precedents show the evidential difference: algorithmic inspiration requires an explicit computational mechanism, while molecular-computing claims require physical materials, protocols, measurements, and controls.

No such wet-lab protocol or biological measurement is present here. DNA-themed forum posts are treated as originating hypotheses, metaphors, or research prompts—not as evidence that software accesses ancestral memory, activates DNA, heals tissue, enhances cognition, or implements organic intelligence.

6. Engineering and numerical risks

Silent domain clamping can hide invalid parameter regimes and map widely different inputs to the same output. Exponential growth can overflow. Step discontinuities make local sensitivity high around zero. Unused parameters can mislead readers into believing the model has dimensions it does not use. Unbounded recursion can diverge or amplify noise; even a bounded recurrence can encode a poor target.

A production-quality successor should validate parameter domains, name step functions accurately, expose numerical warnings, define state and update order, pin seeds for stochastic variants, publish invariants, and compare against simple baselines. Adding mutation terminology without population and fitness semantics would not satisfy this gate.

7. Falsifiable claims and release gates

1. The audit must reproduce all five frozen-simulator properties.
2. The recurrence must pass all five analytical checks and reach the published final state.
3. “Recursive” requires explicit prior-state dependence; pointwise vector evaluation does not qualify.
4. “Genetic algorithm” requires documented representation, population, variation, fitness, selection, and generations.
5. “DNA computing” requires physical or faithfully sourced experimental evidence; software metaphor is insufficient.
6. Health, cognitive, ancestry, healing, or genetic-effect claims require domain expertise, appropriate approvals, and direct evidence and are excluded from this release.

8. Limitations

The audit covers one frozen local simulator revision and an independent mirror, not every similarly named project. The recurrence is intentionally elementary and synthetic. It does not benchmark optimization quality, compare evolutionary algorithms, model biological data, or establish usefulness for AI training. No wet lab, organism, patient, genetic dataset, clinical study, peer review, or independent validation is involved.

9. Conclusion

The strongest form of this research is precise about levels of evidence. The frozen “genetic” program is a piecewise scalar formula with identifiable numerical boundaries. The released recurrence is genuinely recursive and analytically reproducible, but remains a software control model. Keeping metaphor, mathematics, simulation, and biology separate preserves the creative research direction while making every technical claim testable.

Evidence ledger

- Frozen simulator: `_github_video_updates/AgentZERO/big_projects/genetic_adaptation_simulator/simulator.py`, SHA-256
37bab40a3c1af0d03882a1242cbe851944bfe7601e748b8c29440daf22fe7394.
- Frozen README: same directory, SHA-256
30348c1ff37d369a6406bb9dc07faa91907b0d01c6e83c36fc8892b7236b6e94.

- Reproducibility package: `artifacts/bio-inspired-recursive-computation-1.0/`.
- Forum sources are authorial concept notes and are not independent validation.

AI-use disclosure

AI tools assisted corpus search, source inspection, test design, drafting, editing, and packaging. I take responsibility for the claims and release decisions. AI assistance is not peer review or independent validation.

Licensing

The paper is CC BY 4.0. New audit and recurrence code is MIT licensed. Referenced simulator files retain their existing terms and are not redistributed.

References

- [1] J. H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.
- [2] L. M. Adleman. "Molecular Computation of Solutions to Combinatorial Problems." *Science* 266(5187), 1021-1024, 1994. <https://doi.org/10.1126/science.7973651>